OAuth 2.0 Authorization Code Flow Tutorial

By John C. Graham

Overview

This tutorial walks you through implementing OAuth 2.0 authorization code flow to secure API access for a web application. You'll configure an OAuth provider, register your application, and handle the authorization workflow from initial request through token exchange.

Prerequisites

- A registered application with an OAuth 2.0 provider (Google, GitHub, or Auth0)
- Basic understanding of HTTP requests and responses
- Development environment with curl or an HTTP client library

What You'll Build

By the end of this tutorial, your application will:

- Redirect users to an authorization server for consent
- Handle the authorization callback with a secure authorization code
- Exchange the authorization code for access and refresh tokens
- Make authenticated API requests using the access token

Step 1: Register Your Application

Before implementing OAuth, register your application with your chosen provider to obtain client credentials.

1. Navigate to your OAuth provider's developer console
2. Create a new application or project
3. Configure the following settings:
    - **Application name**: Your application's display name
    - **Redirect URI**: `https://yourapp.com/oauth/callback`
    - **Scopes**: Requested permissions (e.g., `read:user`, `write:data`)
4. Save your credentials:

Client ID: abc123xyz789

Client Secret: secret_key_keep_secure

Security Note: Store your client secret securely. Never commit it to version control or expose it in client-side code.

Step 2: Initiate the Authorization Request

Redirect users to the authorization server with the required parameters.

Authorization URL Structure:

```
https://oauth-provider.com/authorize?
  response_type=code&
  client_id=YOUR_CLIENT_ID&
  redirect_uri=https://yourapp.com/oauth/callback&
  scope=read:user write:data&
  state=RANDOM_STATE_STRING
```

Parameter Breakdown:

Parameter	Required	Description
`response_type`	Yes	Must be `code` for authorization code flow
`client_id`	Yes	Your application's client ID
`redirect_uri`	Yes	Must match the URI registered with the provider
`scope`	No	Space-separated list of requested permissions
`state`	Recommended	Random string to prevent CSRF attacks

Example Implementation (Python):

```python
import secrets
from urllib.parse import urlencode

def generate_auth_url(client_id, redirect_uri, scopes):
    state = secrets.token_urlsafe(32)

    # Store state in session for validation
```

```
    session['oauth_state'] = state

    params = {
        'response_type': 'code',
        'client_id': client_id,
        'redirect_uri': redirect_uri,
        'scope': ' '.join(scopes),
        'state': state
    }

    auth_url = f"https://oauth-provider.com/authorize?{urlencode(params)}"
    return auth_url

# Usage
auth_url = generate_auth_url(
    client_id='abc123xyz789',
    redirect_uri='https://yourapp.com/oauth/callback',
    scopes=['read:user', 'write:data']
)

# Redirect user to auth_url
```

Step 3: Handle the Authorization Callback

After the user grants consent, the authorization server redirects back to your callback URI with an authorization code.

Callback URL Format:
```
https://yourapp.com/oauth/callback?
  code=AUTHORIZATION_CODE&
  state=RANDOM_STATE_STRING
```

Validation Steps:

1. **Verify the state parameter** matches the value you generated in Step 2
2. **Extract the authorization code** from the query parameters
3. **Check for error parameters** that indicate denied access

Example Implementation (Python):

```python
from flask import request, session, abort

@app.route('/oauth/callback')
def oauth_callback():
    # Validate state to prevent CSRF
    returned_state = request.args.get('state')
    stored_state = session.get('oauth_state')

    if not returned_state or returned_state != stored_state:
        abort(403, "Invalid state parameter")

    # Check for authorization errors
    error = request.args.get('error')
    if error:
        error_description = request.args.get('error_description', 'Unknown error')
        abort(400, f"Authorization failed: {error_description}")

    # Extract authorization code
    auth_code = request.args.get('code')
    if not auth_code:
        abort(400, "No authorization code received")

    # Proceed to token exchange
    return exchange_code_for_token(auth_code)
```

Step 4: Exchange Authorization Code for Tokens

Exchange the authorization code for access and refresh tokens by making a POST request to the token endpoint.

Token Request Requirements:

```http
POST /oauth/token HTTP/1.1
Host: oauth-provider.com
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code&
code=AUTHORIZATION_CODE&
redirect_uri=https://yourapp.com/oauth/callback&
```

client_id=YOUR_CLIENT_ID&

client_secret=YOUR_CLIENT_SECRET

Example Implementation (Python):

python
```python
import requests

def exchange_code_for_token(auth_code):
    token_url = "https://oauth-provider.com/oauth/token"

    payload = {
        'grant_type': 'authorization_code',
        'code': auth_code,
        'redirect_uri': 'https://yourapp.com/oauth/callback',
        'client_id': 'abc123xyz789',
        'client_secret': 'secret_key_keep_secure'
    }

    response = requests.post(token_url, data=payload)

    if response.status_code != 200:
        raise Exception(f"Token exchange failed: {response.text}")

    token_data = response.json()

    return token_data
```

Expected Response:

json
```json
{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "refresh_token": "refresh_token_string",
  "scope": "read:user write:data"

}
```

Security Best Practice: The authorization code is single-use and expires quickly (typically 10 minutes). Exchange it immediately after receiving it.

Step 5: Make Authenticated API Requests

Use the access token to authenticate requests to protected resources.

Request Format:

http
```
GET /api/user/profile HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...
```

Example Implementation (Python):

python
```python
def get_user_profile(access_token):
    api_url = "https://api.example.com/user/profile"

    headers = {
        'Authorization': f'Bearer {access_token}',
        'Accept': 'application/json'
    }

    response = requests.get(api_url, headers=headers)

    if response.status_code == 401:
        # Token expired, refresh it
        return refresh_access_token()

    if response.status_code != 200:
        raise Exception(f"API request failed: {response.text}")

    return response.json()
```

Step 6: Handle Token Refresh

Access tokens expire. Use the refresh token to obtain a new access token without user interaction.

Refresh Token Request:

http
```
POST /oauth/token HTTP/1.1
Host: oauth-provider.com
```

```
Content-Type: application/x-www-form-urlencoded

grant_type=refresh_token&
refresh_token=REFRESH_TOKEN&
client_id=YOUR_CLIENT_ID&
client_secret=YOUR_CLIENT_SECRET
```

Example Implementation (Python):

```python
def refresh_access_token(refresh_token):
    token_url = "https://oauth-provider.com/oauth/token"

    payload = {
        'grant_type': 'refresh_token',
        'refresh_token': refresh_token,
        'client_id': 'abc123xyz789',
        'client_secret': 'secret_key_keep_secure'
    }

    response = requests.post(token_url, data=payload)

    if response.status_code != 200:
        # Refresh token expired, require re-authentication
        return redirect_to_login()

    new_token_data = response.json()

    # Update stored tokens
    save_tokens(new_token_data)

    return new_token_data['access_token']
```

Troubleshooting

Error: "invalid_grant"

- **Cause**: Authorization code already used or expired
- **Solution**: Authorization codes are single-use. Generate a new authorization request.

Error: "redirect_uri_mismatch"

- **Cause**: The redirect URI in your token request doesn't match the registered URI

- **Solution**: Ensure the redirect URI exactly matches your provider configuration, including protocol (https://) and trailing slashes.

Error: "invalid_client"

- **Cause**: Client ID or secret is incorrect
- **Solution**: Verify your credentials match those registered with the OAuth provider.

401 Unauthorized on API requests

- **Cause**: Access token expired or invalid
- **Solution**: Refresh the access token using your refresh token, or re-authenticate the user.

Security Checklist

✅ Store client secrets securely (environment variables, secret management service)
✅ Validate the `state` parameter to prevent CSRF attacks
✅ Use HTTPS for all OAuth URLs and callbacks
✅ Set appropriate token expiration times
✅ Implement token refresh before expiration
✅ Never expose tokens in URLs or logs
✅ Revoke tokens when users log out

Next Steps

- Implement token storage securely (encrypted database or secure session)
- Add error handling for network failures and rate limits
- Implement automatic token refresh before expiration
- Add logging for authorization events (without logging token values)
- Test the complete flow in a staging environment

Additional Resources

- RFC 6749: OAuth 2.0 Authorization Framework
- OAuth 2.0 Security Best Practices